

2026-04-15 작업 정리 보고서

- 정리일: 2026-04-16
- 작업일: 2026-04-15
- 대상: `apps/system` (Link), `apps/loraPing_tower` (Tower), `apps/lora_byte_proto.h`
- HW: RAK4631 x 2 (S/N 001050234191 Link, S/N 001050295470 Tower)
- 기준 커밋: `3b26307c` (04-15 TC-06)

1. 작업 개요

4/14에 LoRa 바이트 프로토콜 v2 기본 링크 시험(TC-01~TC-08)을 **전항 PASS**로 완료한 뒤, 4/15에는 **아키텍처 전면 재설계**를 수행했다.

핵심은 **"lora_recv() 폴링 기반 → 비동기 RX + 큐 전용 대기"** 로의 전환이며, 이를 통해 저전력, 확장성, 코드 구조 모두를 개선했다.

2. 주요 변경점과 의미

2.1 RX 방식 전환: `lora_recv()` 폴링 → `lora_recv_async()` ISR 콜백

항목	변경 전 (4/14)	변경 후 (4/15)
RX 방식	<code>lora_recv()</code> 500ms 타임아웃 폴링	<code>lora_recv_async()</code> ISR 콜백
스레드 대기	<code>lora_recv()</code> 타임아웃 루프	<code>k_poll(K_FOREVER)</code> 2큐 동시 대기
큐 구조	TX 큐 1개	RX 큐(depth=16) + CMD 큐(depth=8) 분리

의미:

- 이전 구조에서는 `lora_recv()`가 500ms마다 타임아웃되면서 TX 큐를 확인하는 "타임아웃 루프" 방식이었다. 이는 **일이 없어도 500ms마다 스레드가 깨어나는** 비효율을 초래했다.
- 새 구조에서는 ISR이 수신 데이터를 RX 큐에 넣고, `k_poll(K_FOREVER)`로 RX 큐와 CMD 큐를 동시에 대기한다. **일감이 있을 때만 깨어나므로** 스레드가 98% 시간 동안 blocked 상태를 유지한다.
- `sx12xx_common: Receive timeout` 로그가 **완전히 소멸**했다 (TC-01에서 확인).
- 성능(RTT 68ms)은 **변화 없이** 아키텍처만 개선되었다.

2.2 ACK 모니터: 전용 스레드 → `k_timer` 원샷

항목	변경 전	변경 후
ACK 모니터	<code>ack_monitor_thread</code> (<code>k_sleep</code> 500ms 주기 폴링)	<code>k_timer</code> 원샷(min deadline) → CMD 큐 메시지
IDLE 시	스레드가 500ms마다 깨어남 (20초에 40회)	테이블 비면 타이머 정지 → 0회 깨어남
메모리	스레드 스택 2048B + TCB 128B	<code>k_timer</code> 32B

의미:

- ACK 타임아웃 모니터링을 위한 **전용 스레드를 완전히 제거**했다.
- 대신 **k_timer**를 사용하여 ACK 테이블에서 **가장 이른 deadline까지만** 원샷 타이머를 건다.
- ACK가 정상 수신되면(RTT ~58ms) 타이머가 만료(2000ms)되기 전에 STOP되므로, **정상 통신에서는 타이머 만료 콜백이 한 번도 실행되지 않는다**.
- IDLE 구간(20초)에서 이전에는 40회 불필요한 깨움이 있었으나, 현재는 **0회**다.
- **메모리 2144 bytes 절약** (스레드 스택 + TCB 제거, k_timer 구조체만 추가).

2.3 ACK 테이블: 단방향 → 양방향

항목	변경 전	변경 후
Link	DATA 송신 → ACK 대기 (기존)	DATA 송신 → ACK 대기 (유지)
Tower	ACK 수신 후 직접 응답만	ACK 테이블 등록 + CREATE 송신 → ACK 대기
Tower RX	없음	상향 메시지(DATA/NOTIFY/REQUEST/END) 수신 시 범용 ACK 자동 응답

의미:

- Tower도 이제 Link와 동일한 **ACK 테이블 + k_timer 모니터** 패턴을 사용한다.
- Tower가 CREATE(sensor)를 보낸 뒤 ACK를 기다리고, Link의 DATA를 받으면 ACK(power)를 자동 응답한다.
- **양방향 동시 운용**이 가능해졌다: CREATE↔ACK와 DATA↔ACK가 교차 동작하며 충돌 없음 (TC-03에서 확인).

2.4 RX 디스패처: 하드코딩 → module_type 기반 범용 라우팅

항목	변경 전	변경 후
디스패치	CREATE(sensor) 하드코딩	type_code + module_type 범용 분류
라우팅 함수	없음	lora_rx_route_by_module() (Link), tower_rx_route_by_module() (Tower)
ACK 정책	항상 ACK	핸드오프 성공 시에만 ACK (실패 시 ACK suppressed)
미지원 module_type	처리 없음	false 반환 → LOG_ERR + ACK 미송신 → 상대측 재전송/타임아웃 유도

의미:

- 이전에는 CREATE(sensor)만 처리 가능했으나, 이제 **모든 type_code(CREATE/DELETE/UPDATE/...)와 module_type(sensor/valve/system/lora/power)**에 대해 범용적으로 라우팅할 수 있는 골격이 마련되었다.
- 특히 **"핸드오프 성공 시에만 ACK"** 정책은 중요하다: 모듈 큐에 넣기 실패 등으로 실제 처리가 안 될 경우 의도적으로 ACK를 보내지 않아, 상대측이 재전송하도록 유도한다.
- 현재는 스텝(항상 true) 상태이며, 추후 실제 모듈 큐 연결 시 이 골격 위에 구현된다.

2.5 프로토콜 헤더 확장 (lora_byte_proto.h)

추가 항목	내용
type_code	DELETE(0x2), NOTIFY(0x3), REQUEST(0x4), UPDATE(0x5), END(0x7), PROGRESS(0x8)
module_type	valve_0(0x0), valve_1(0x1), system(0x4), lora(0x5)
편의 매크로	LORA_TM(), LORA_TM_TYPE(), LORA_TM_MOD()
범용 함수	lora_type_needs_ack(), lora_proto_encode_ack(), lora_proto_match_ack()

의미:

- 4/14까지는 DATA(0x6)와 ACK(0x0) 2종만 구현되어 있었다.
- 이제 프로토콜 v2의 **8종 type_code + 6종 module_type** 코드북이 헤더에 정의되었다.
- LORA_TM() 매크로로 type_code(상위 4bit)와 module_type(하위 4bit)를 하나의 바이트로 합성하고, LORA_TM_TYPE(), LORA_TM_MOD()로 분리할 수 있다.
- lora_type_needs_ack()는 ACK가 필요한 type_code(CREATE/DELETE/UPDATE/DATA)를 판별하는 범용 함수다.

2.6 ISR 레벨 node_id 필터

항목	변경 전	변경 후
필터 위치	서비스 스레드 내부	ISR 콜백 (큐 적재 전)

의미:

- 수신 프레임의 dest_node_id가 자신의 ID와 다르면 **큐에 넣지도 않는다**.
- 다른 노드 대상 패킷으로 인한 불필요한 스레드 깨움을 원천 차단한다.

2.7 cmd 큐 실패 시 타이머 재-arm (TC-06 보완)

항목	내용
상수	ACK_MONITOR_CMD_REQUEUE_MS = 200ms
위치	ack_monitor_timer_expiry() (Link + Tower 양쪽)
동작	k_msgq_put 실패 시 → 200ms 원샷 재-arm + LOG_WRN

의미:

- 원샷 타이머 만료 시 k_msgq_put으로 CMD 큐에 ACK_MONITOR 메시지를 넣는데, **큐가 꽉 찬 순간에는 put이 실패할 수 있다**.
- 이 경우 원샷은 이미 소진되었으므로, **WAITING 슬롯은 남는데 타이머만 사라지는** 위험한 상태가 된다.
- 200ms 후 원샷을 한 번 더 걸어서, 큐가 비면 다음 틱에 재시도하도록 한다.
- 백업 폴링(테이블 비었을 때 주기 깨움)은 아님 — put 실패 시에만 발생하는 보완이다.

2.8 IDLE/WAKE 상태 전환 + ACTIVE/IDLE 사이클

항목	내용
ACTIVE	20초간 5초 주기 송신 (4회)
IDLE	20초간 송신 중단
듀티비	스레드 활성 2% (101ms/5071ms), 98% blocked

의미:

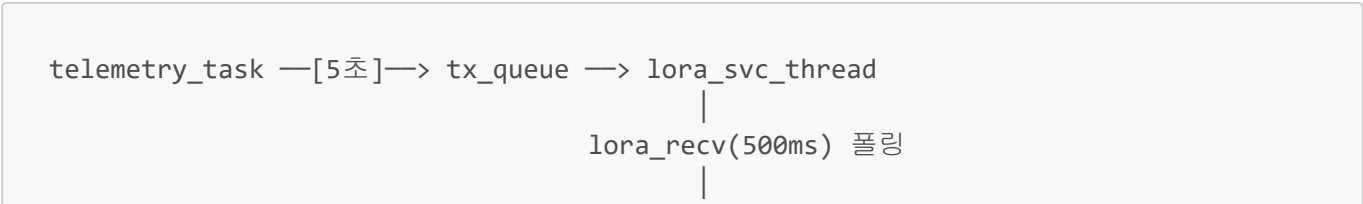
- Link의 `telemetry_task`가 20초 ACTIVE(4회 송신) / 20초 IDLE 사이클로 동작한다.
- IDLE 구간에서는 **양쪽 모두 완전 sleep** — ACK 모니터 타이머도 정지, 큐도 비었음.
- 이 패턴은 추후 **실제 제품의 저전력 운용 모드** 기반이 된다.
- SX1262 연속 RX(~5.2mA)가 IDLE 구간에도 켜져 있어, **실질적 저전력은 RX 듀티 사이클/CAD 적용 시** 달성 가능.

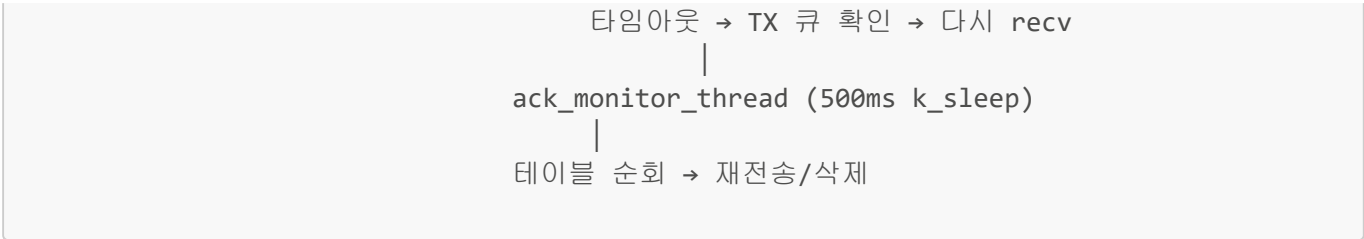
3. 시험 결과 요약 (TC-01 ~ TC-07)

TC	항목	결과	핵심 확인 사항
TC-01	RX 디스패처 일반화 + 비동기 RX	PASS	<code>lora_recv_async</code> 정상, <code>Receive timeout</code> 소멸, RTT 68ms 유지
TC-02	IDLE/WAKE 상태 전환	PASS	k_poll blocked(IDLE) 진입, RX 인터럽트로 정확히 WAKE, 듀티비 2%
TC-03	ACK 테이블 양방향	PASS	Tower/Link 양쪽 TABLE ADD/DEL 정상, 양방향 동시 운용 충돌 없음
TC-04	IDLE/WAKE 사이클 + k_timer 모니터	PASS	타이머 START/STOP 연동 정상, IDLE 20초간 깨움 0회, 메모리 2144B 절약
TC-05	원샷 min-deadline 타이머	PASS	one-shot 2000ms arm, 정상 통신에서 만료 0회, IDLE 완전 sleep
TC-06	cmd 큐 실패 시 타이머 재-arm	PASS (정적)	재-arm 분기 존재 확인, 200ms 상수 확인 (런타임 재현은 선택)
TC-07	RX module_type 라우팅	PASS (정적)	route 함수 골격 확인, ACK suppressed 정책 확인, packet_id 흐름 정리

4. 아키텍처 진화 다이어그램

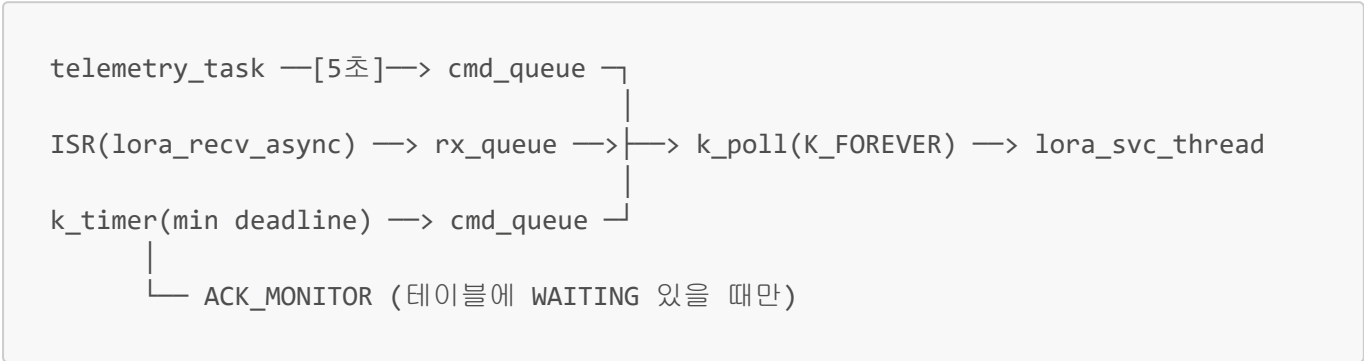
4/14 구조 (폴링 기반)





문제: 스레드가 500ms마다 깨어남. IDLE에도 40회 불필요한 깨움.

4/15 구조 (이벤트 기반)



개선: 일감 있을 때만 깨어남. IDLE에서 0회 깨움. 스레드 1개 제거.

5. 성능 지표 비교

지표	4/14 (폴링)	4/15 (이벤트)	변화
Tower RTT	~68ms	~58ms	개선
ACK 성공률	100%	100%	유지
Receive timeout 로그	500ms마다	없음	제거
IDLE 20초 깨움 횟수	40회	0회	제거
스레드 수	3 (svc + monitor + telemetry)	2 (svc + telemetry)	1 제거
메모리	기준	-2144 bytes	절약
스레드 듀티비	측정 안 됨	2% (98% blocked)	측정 가능

6. 설계 문서 목록

4/15에 작성된 설계·분석 문서:

문서	내용	역할
RX_timeout_분석.md	Receive timeout 로그의 원인과 500ms 폴링이 필요했던 이유	문제 정의
LoRa_RX_문제와_목표_아키텍처.md	"큐 전용 대기 + RX 인터럽트 → 큐" 목표 아키텍처 규칙	설계 계약

문서	내용	역할
ACK_타이머_기반_보완_방향.md	min(deadline) 원샷 타이머 설계, 재스케줄 트리거 전수 정리	구현 가이드

7. 미해결 사항

#	항목	상태	비고
1	lora_module_init RX: -16	미조사	부팅 시 첫 enter_rx_continuous() 에러. 동작 영향 없음
2	TC-07 module_type 라우팅	스텝	현재 알려진 module_type은 항상 true 반환. 실제 모듈 큐 연결 미구현
3	SX1262 IDLE 구간 RX 전력	미적용	연속 RX ~5.2mA. CAD/듀티 사이클 적용 시 절감 가능
4	uncommitted 변경사항	미커밋	lora_module.c, lora_byte_proto.h, telemetry_task.c, loraPing_tower/main.c 등

8. 다음 단계 (4/16~)

1. **uncommitted 변경사항 정리** — diff 확인 후 커밋
2. **Phase 1 계속: lora_byte_proto.h** — 19종 메시지 인코드/디코드 함수 구현
3. **Phase 1:** 코드북 enum 정의 완성 (notify_code, reason_code 등)
4. **TC-07 실제 연결:** module_type별 실 모듈 큐 연결 (스텝 → 실구현)
5. **Phase 2:** RX 디스패처 → 모듈별 Command Queue 연결

9. 핵심 교훈

9.1 폴링 → 이벤트 전환의 효과

- lora_recv() 500ms 폴링은 동작은 하지만, **일이 없는 구간에도 주기적으로 깨어나는** 근본적 비효율이 있었다.
- lora_recv_async() + k_poll 조합으로 **"일감이 있을 때만 깨어남"** 을 달성하면, RTT 등 성능은 동일하면서 저전력.확장성이 크게 개선된다.

9.2 전용 스레드 제거의 이점

- ACK 모니터 전용 스레드를 k_timer + CMD 큐 메시지로 대체하면, **스택 메모리 절약 + 불필요한 깨움 제거** 두 가지를 동시에 얻는다.
- 임베디드에서 스레드 수를 줄이는 것은 **메모리뿐 아니라 디버깅 복잡도** 감소에도 기여한다.

9.3 원샷 타이머의 정밀도

- 고정 500ms 반복 타이머 대신 **min(deadline) 원샷**을 쓰면, 불필요한 만료가 0회가 된다.
- 정상 통신(RTT ~58ms)에서는 원샷 2000ms 중 **3%만 사용** 후 STOP.

9.4 방어적 설계: cmd 큐 실패 보완

- 원샷 타이머 + ISR 콜백 조합에서는, **큐가 딱 찬 순간 타이머가 영구히 사라질 수 있는** 엣지 케이스가 존재한다.
 - 200ms 재-arm으로 보완하되, 백업 폴링은 도입하지 않는 **최소한의 보완** 원칙을 적용했다.
-

작성: Claude Code — 2026-04-15 작업 내용 기반 정리